



Lab Report 6: Stevie and the Spark Snufflers

Team Gamma
Nick, Daniel, Aziz, Kevin
12/02/24

Introduction and Objectives

In this lab, expand on our completed robots from the semester. We aim to create firefighting robots which will require the integration of new components such as photoresistors, thermistors, fans, LED lights, and buzzers. Additionally, steps will be taken to fabricate mounts for the sensors and add Firehouse Subs hats to the robot chassis.

Goals:

- Build 4 firefighting robots
- Detect and follow fire
- Put out fire via fan
- Activate fire mode (sirens and lights)

Objectives:

- Construct independent systems for photoresistor, thermistor, LEDs, buzzer, and motor relay
- Detect light source within the ambient light environment
- The relay circuit should be activated by 5v Arduino logic and power on and off the motor fan
- Thermistor levels activate and deactivate the relay circuit
- Fire mode is activated when potential fire is detected (certain light level)

Methods and Results

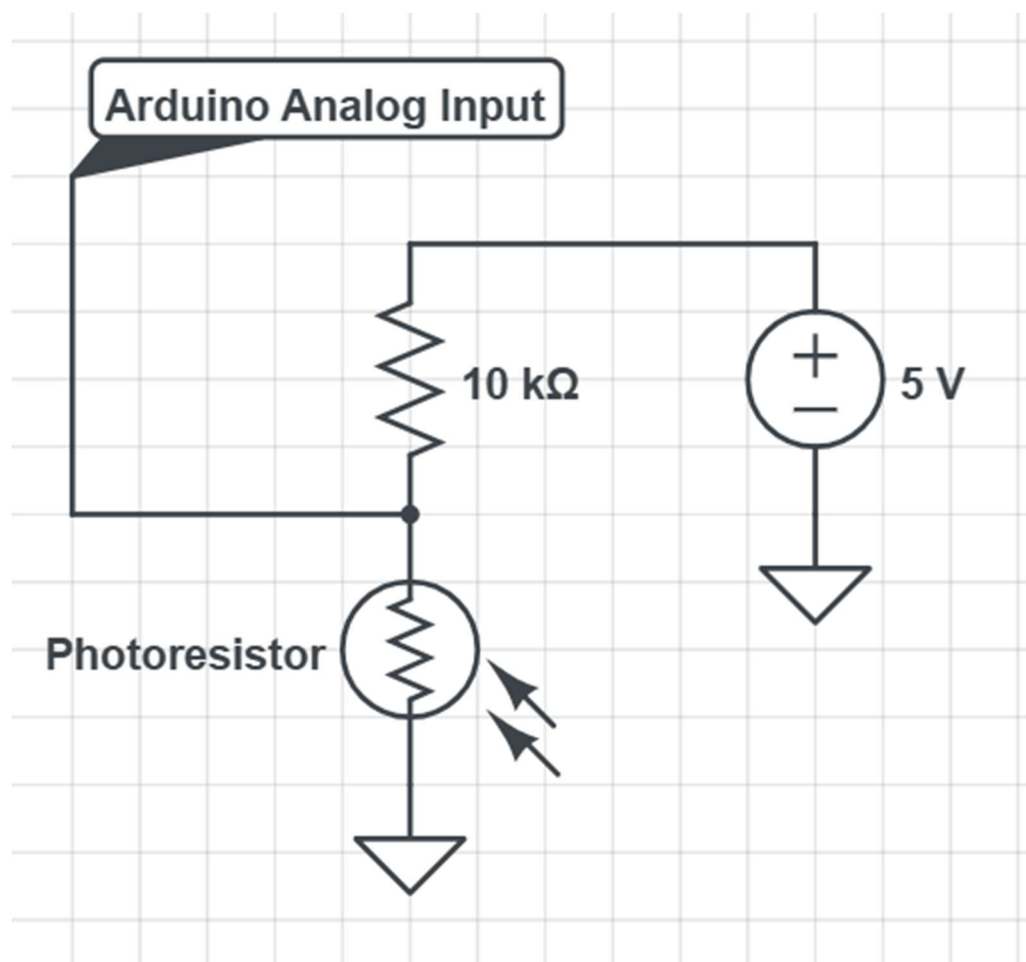
Equipment and Materials:

- Breadboard and jumper wires
- Resistors and capacitors
- Robot kit
- Previous circuit
- Arduino Nano Every

- Arduino software
- x2 photoresistors
- x1 passive buzzer
- x1 10,000 rpm motor and fan
- x1 relay
- x1 thermistor
- CAD software
- 3D printer
- x1 Firehouse Subs chief hat
- Glue and duct tape
- Soldering equipment

Photoresistor Circuit

To begin, a photoresistor circuit was developed to enable the robot to detect varying light levels:

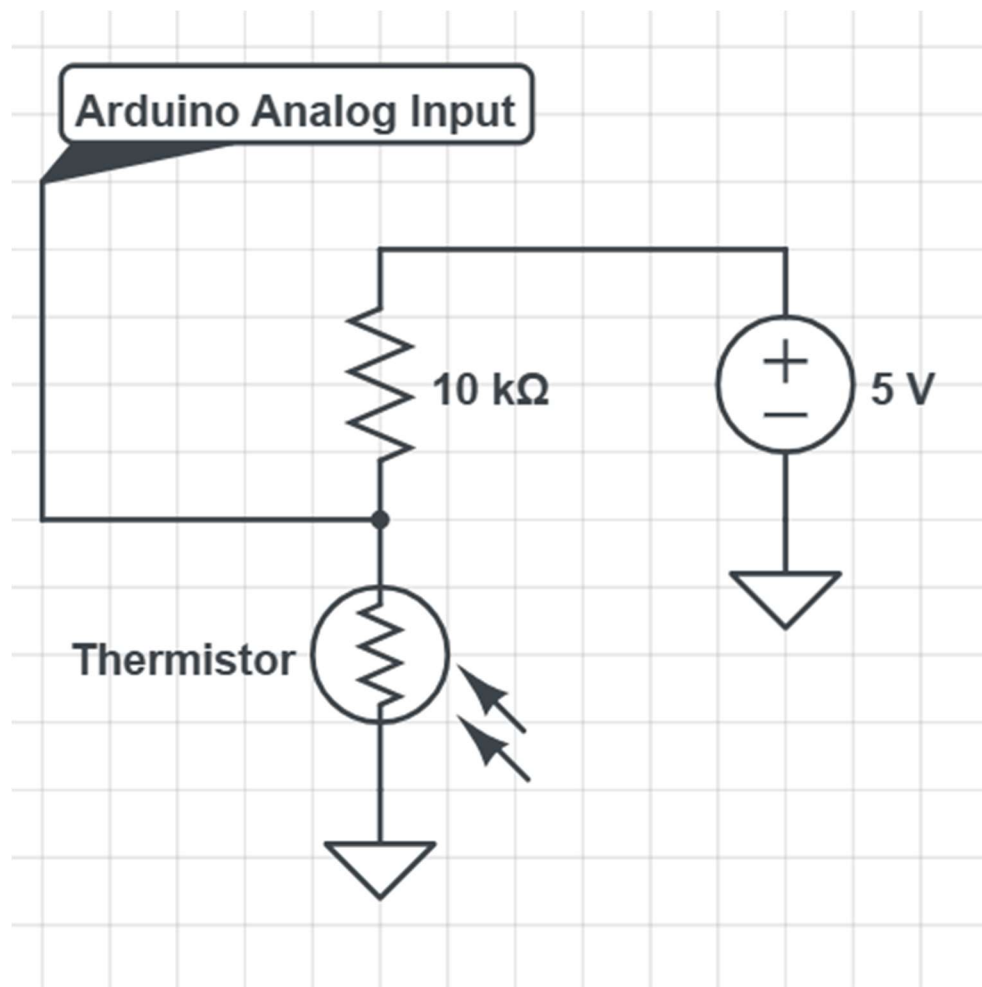


The robot was equipped with two of these circuits to allow the robot to compare values of a left-angled sensor and a right-angled sensor to assist in tracking a light/flame.

Note: The Arduino provided the 5v source.

Thermistor Circuit

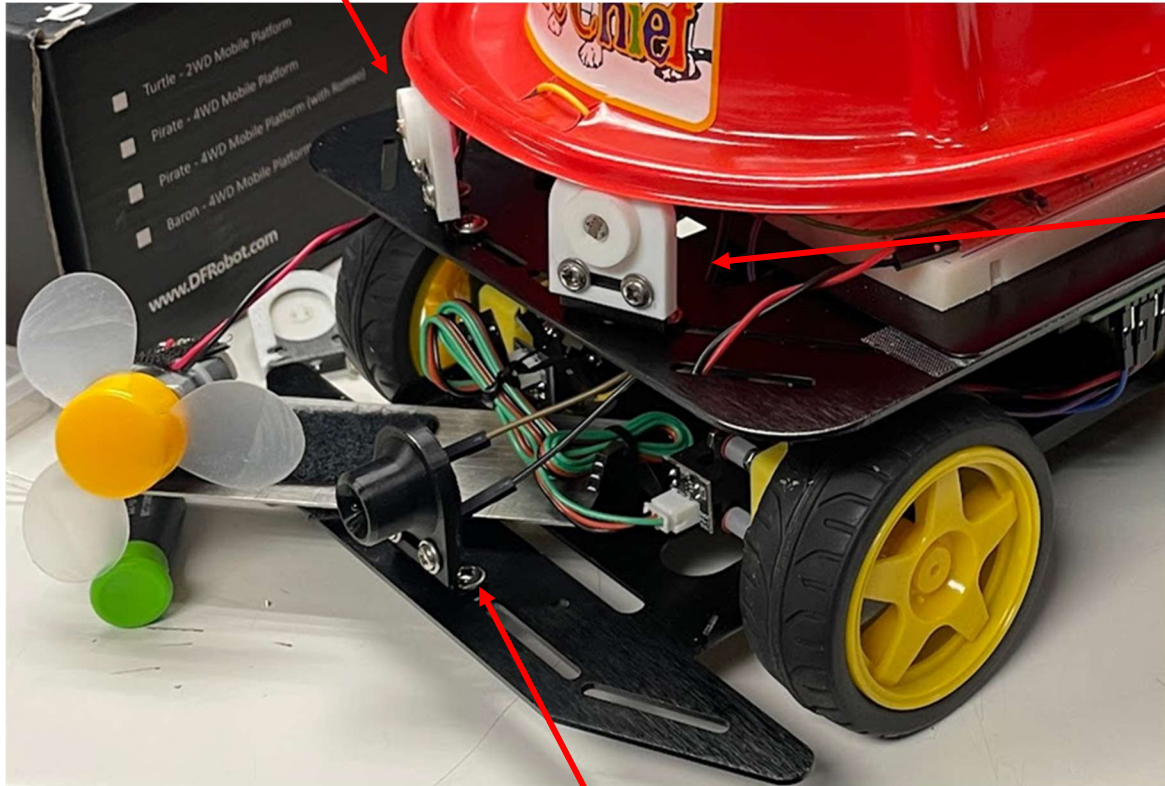
A similar circuit was built for the thermistor:



Only one of these circuits was built with its intended purpose being detecting changes of heat in the environment. With this, we can identify whether the light source is a fire and active countermeasures.

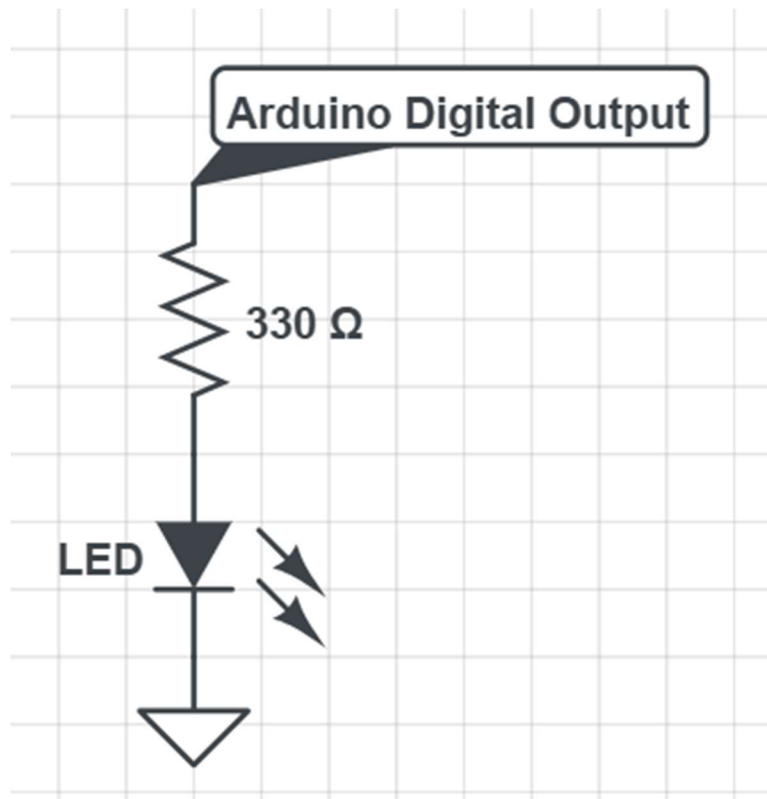
Thermistors and photoresistors were mounted to the chassis using metal brackets and screws provided in the kit, along with 3D-printed components to slide over the brackets and securely hold electronic components:

3 sensor mounts



LED Firemode Lights Circuit

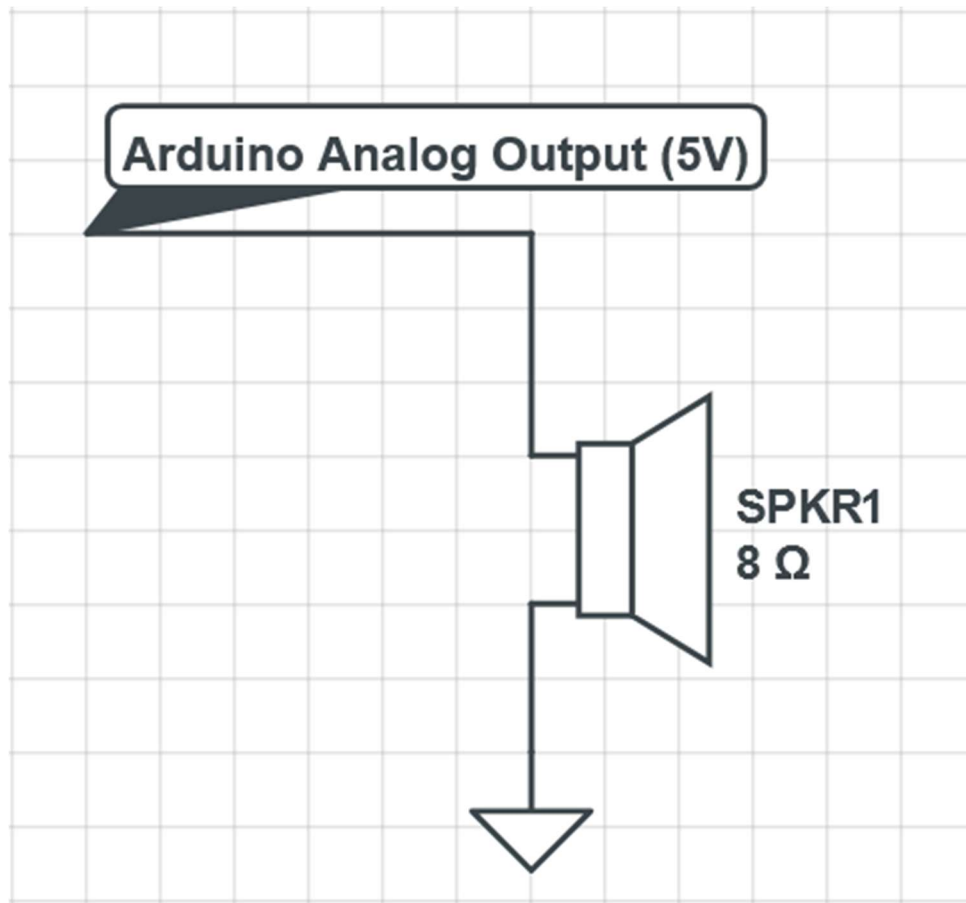
The robot should have some level of emergency lights to indicate when it is in pursuit of a possible fire. To do this, we implemented 4 of the following circuit:



The LEDs were alternated between 2 different colors on each of the robots.

Buzzer Firemode Siren Circuit

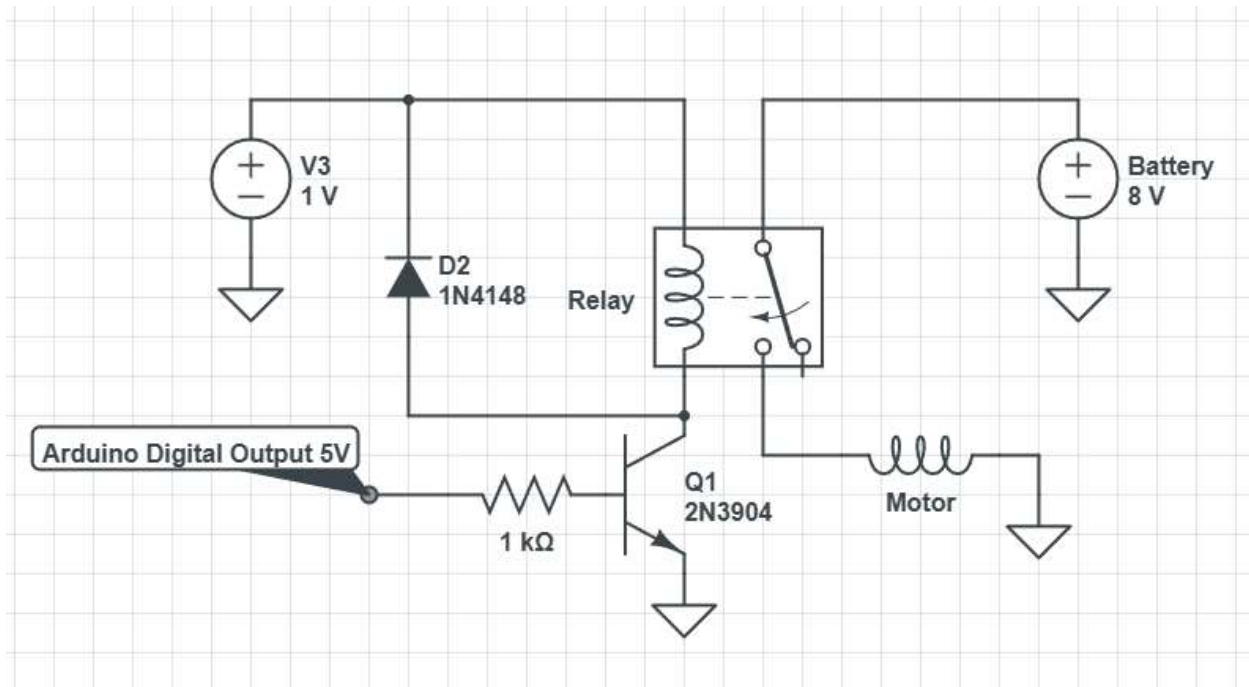
Along with the emergency lights, the following buzzer circuit was added to act as a siren:



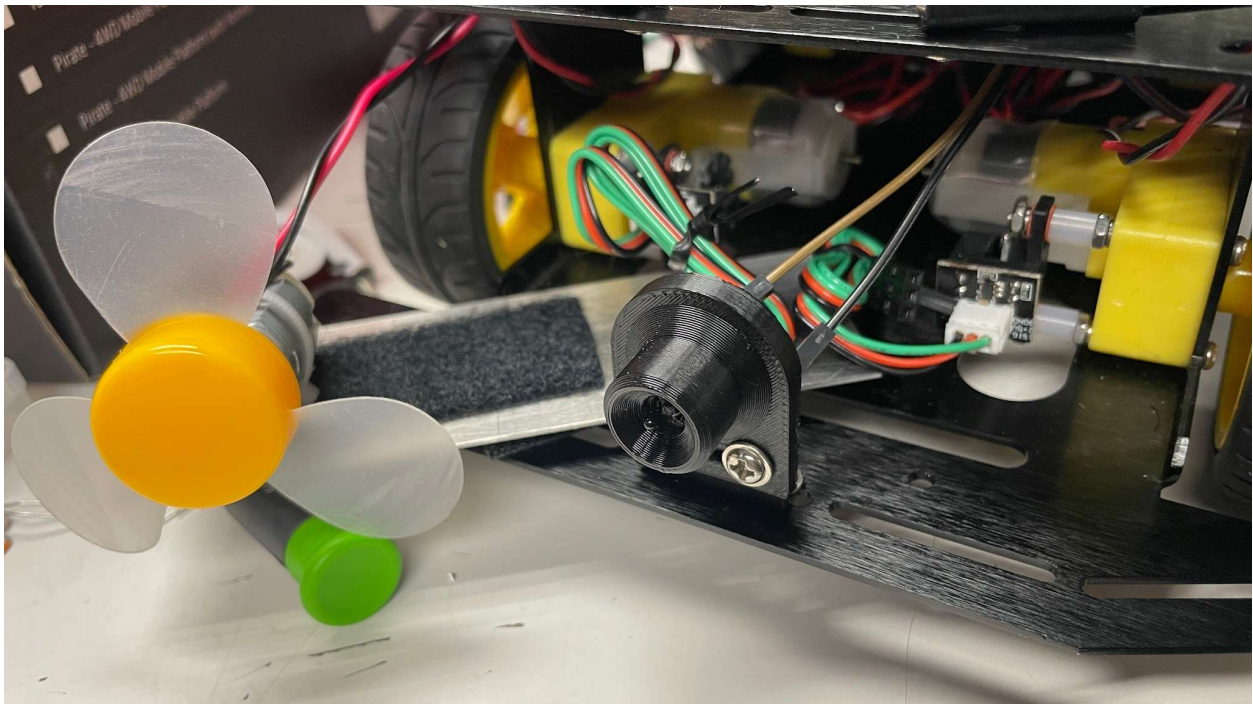
Note: The component pictured here is a buzzer and not a speaker.

Relay Circuit and Motor Fan Integration

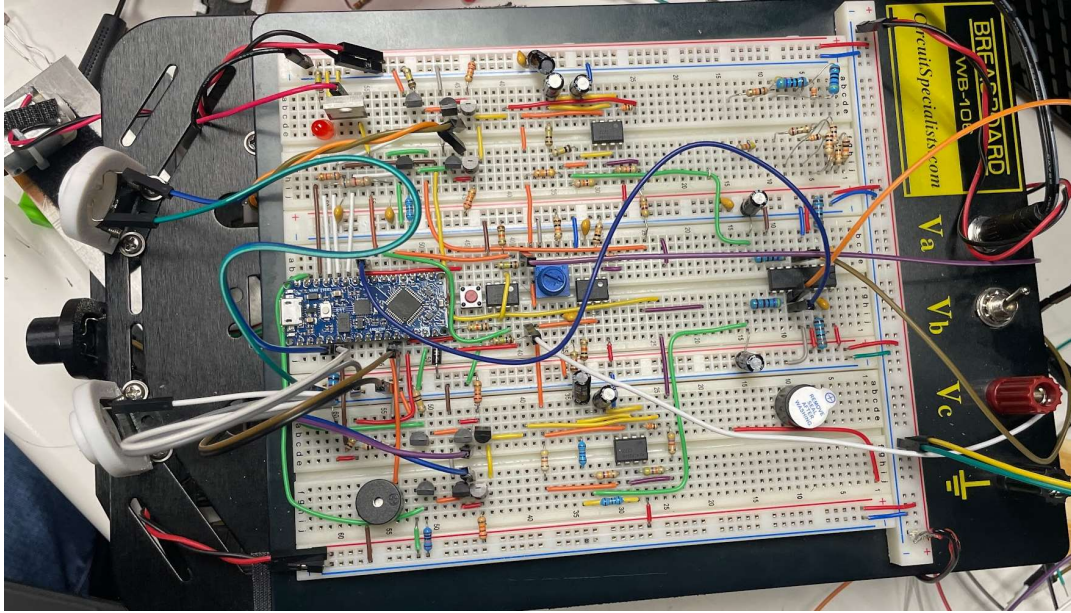
Most importantly, the robot needed a method of extinguishing fire. To do this a motor was used to spin a fan that would blow out flames. The following circuit was developed and implemented:



Here, a fan was connected to the motor in this circuit like so:



The overall circuit with all new components integrated can be seen below:



Code and Testing

A couple of overall code iterations were developed for the robots to accomplish the same task which can be found below. Both start the robot off by searching for a light source, upon finding a light source pursue it and activate fire mode (lights and sirens), and upon confirming it is a fire (thermistor temperature), extinguish it with onboard countermeasures (DC motor powered fan).

Nick's Code:

```
// Pin setup
//pin 5
const int LForward = 11;
const int LBackward = 12;
const int LPWM = 10;
const int RForward = 7;
const int RBackward = 8;
const int RPWM = 9;
const int buzzer = 17;
const int Relay = 5;

int ThermPin = 16;
int Vo;
float R1 = 10000;
float logR2, R2, T;
```

```
float c1 = 1.009249522e-03, c2 = 2.378405444e-04, c3 = 2.019202697e-07;
```

```
int PhotoRes1 = A1;
```

```
int PhotoRight = 0;
```

```
int PhotoRes2 = A0;
```

```
int PhotoLeft = 0;
```

```
int AmbientLight = 600;
```

```
int DifThresh = 50;
```

```
int count = 0;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    pinMode(LForward, OUTPUT);
```

```
    pinMode(LBackward, OUTPUT);
```

```
    pinMode(LPWM, OUTPUT);
```

```
    pinMode(RForward, OUTPUT);
```

```
    pinMode(RBackward, OUTPUT);
```

```
    pinMode(RPWM, OUTPUT);
```

```
    pinMode(21, OUTPUT);
```

```
    pinMode(20, OUTPUT);
```

```
    pinMode(19, OUTPUT);
```

```
    pinMode(18, OUTPUT);
```

```
    pinMode(buzzer, OUTPUT);
```

```
    pinMode(Relay, OUTPUT);
```

```
    digitalWrite(LForward, LOW); // Stop forward
```

```
    digitalWrite(LBackward, LOW); // Stop backward
```

```
    digitalWrite(RForward, LOW); // Stop forward
```

```
    digitalWrite(RBackward, LOW); // Stop backward
```

```
    analogWrite(LPWM, 225); // Set Vref, duty cycle 204/255
```

```
    analogWrite(RPWM, 201); // Set Vref, duty cycle 201/255
```

```
    digitalWrite(21, LOW); // LED off
```

```
    digitalWrite(20, LOW); // LED off
```

```
    digitalWrite(19, LOW); // LED off
```

```
    digitalWrite(18, LOW); // LED off
```

```
    digitalWrite(Relay, LOW); // Fan off
```

```
    delay(3000);
```

```
}
```

```
void loop() {
```

```

Serial.print("Left Ball: ");
Serial.println(PhotoLeft);
Serial.print("Right Ball: ");
Serial.println(PhotoRight);

delay(275);
Stop(); // Reset motors
digitalWrite(Relay, LOW); // Fan off

Vo = analogRead(ThermPin);
R2 = R1 * (1023.0 / (float)Vo - 1.0);
logR2 = log(R2);
T = (1.0 / (c1 + c2 * logR2 + c3 * logR2 * logR2 * logR2));
T = T - 273.15;
T = (T * 9.0) / 5.0 + 32.0;

PhotoRight = analogRead(PhotoRes1);

PhotoLeft = analogRead(PhotoRes2) + 20;

int dif = abs(PhotoRight - PhotoLeft);

if (PhotoRight > PhotoLeft && PhotoLeft < AmbientLight && dif >
DifThresh) {
    analogWrite(LPWM, 225); // Set Vref, duty cycle 204/255
    analogWrite(RPWM, 201); // Set Vref, duty cycle 201/255
    Left();
    Serial.println("Something to the Left");
    FireMode();
}

if (PhotoRight < PhotoLeft && PhotoRight < AmbientLight && dif >
DifThresh) {
    analogWrite(LPWM, 225); // Set Vref, duty cycle 204/255
    analogWrite(RPWM, 201); // Set Vref, duty cycle 201/255
    Right();
    Serial.println("Something to the Right");
    FireMode();
}

if (dif < DifThresh && PhotoLeft < AmbientLight) {

```

```

    analogWrite(LPWM, 225); // Set Vref, duty cycle 204/255
    analogWrite(RPWM, 201); // Set Vref, duty cycle 201/255
    Forward();
    Serial.println("Straight ahead!");
    FireMode();
}

if (dif < DifThresh && PhotoLeft < 60) {
    analogWrite(LPWM, 225); // Set Vref, duty cycle 204/255
    analogWrite(RPWM, 201); // Set Vref, duty cycle 201/255
    Reverse();
    Serial.println("Too close! Hot!");
    FireMode();
}

if (dif < DifThresh && PhotoLeft >= AmbientLight) {
    analogWrite(LPWM, 125); // Set Vref, duty cycle 204/255
    analogWrite(RPWM, 101); // Set Vref, duty cycle 201/255
    noTone(buzzer); // Stop sound
    digitalWrite(21, LOW);
    digitalWrite(20, LOW);
    digitalWrite(19, LOW);
    digitalWrite(18, LOW);
    Left();
    Serial.println("Looking");
}
}

// Movement Functions

void FireMode() {
    if (count == 0) {
        //Serial.println("Switch1");
        digitalWrite(18, HIGH);
        digitalWrite(21, HIGH);
        digitalWrite(20, LOW);
        digitalWrite(19, LOW);
        tone(buzzer, 1150); // Send sound signal
        count = 1;

        if (T > 75) {

```

```

        digitalWrite(Relay, HIGH); // Fan on
        Serial.println("Fire! Activating countermeasures!");
    }

    else {
        digitalWrite(Relay, LOW); // Fan off
    }
}

else {
    //Serial.println("Switch2");
    digitalWrite(18, LOW);
    digitalWrite(21, LOW);
    digitalWrite(20, HIGH);
    digitalWrite(19, HIGH);
    tone(buzzer, 950); // Send sound signal
    count = 0;

    if (T > 75) {
        digitalWrite(Relay, HIGH); // Fan on
        Serial.println("Fire! Activating countermeasures!");
    }

    else {
        digitalWrite(Relay, LOW); // Fan off
    }
}
}

void Forward() {
    digitalWrite(LForward, HIGH); // Left side forward
    digitalWrite(RForward, HIGH); // Right side forward
}

void Reverse() {
    digitalWrite(LBackward, HIGH); // Left side forward
    digitalWrite(RBackward, HIGH); // Right side forward
}

void Stop() {
    digitalWrite(LForward, LOW); // Stop forward
    digitalWrite(LBackward, LOW); // Stop backward
    digitalWrite(RForward, LOW); // Stop forward
}

```

```

    digitalWrite(RBackward, LOW); // Stop backward
}

void Right() {
    digitalWrite(LBackward, HIGH); // Left side backward
    digitalWrite(RForward, HIGH); // Right side forward
}

void Left() {
    digitalWrite(LForward, HIGH); // Left side forward
    digitalWrite(RBackward, HIGH); // Right side backward
}

```

Kevin's Code:

```

#include <math.h>

//Setting up pins : Left side pins first then right side
const int pinForward[] = {11,7};
const int pinBackward[] = {12,8};
const int pinPWM[] = {10,9};
const int pinEnc[] = {3,2};
//const int pinEncB[] = {5,4}; -----comment out bc no need for enc pos or dir
const int fanPin = 4;
volatile int posi[] = {0,0};
const int pinON = 6;

//other globals to set up
float tar_pulses[] = {0,0};
long prevT = 0;
int noise = 1; // buzzer initialization
int thermMax=470;

void setup() {
    Serial.begin(9600); //serial port
    pinMode(pinON, INPUT_PULLUP); //button setup pullup = high
    pinMode(LED_BUILTIN, OUTPUT); // onboard led for signifying things
    pinMode(A6, OUTPUT); //buzzer pin
    pinMode(fanPin, OUTPUT);
}

```

```

for(int k = 0; k < 2 ; k++){//setting up motor pins
    pinMode(pinForward[k], OUTPUT);
    pinMode(pinBackward[k], OUTPUT);
    pinMode(pinPWM[k], OUTPUT);
    //pinMode(pinEncB[k], INPUT); -----
}
attachInterrupt(digitalPinToInterrupt(pinEnc[0]), readEncoder<0, -1>,
RISING);//cw increase
attachInterrupt(digitalPinToInterrupt(pinEnc[1]), readEncoder<1, 1>,
RISING);//cw decrease

}

void loop() {
    // full reset
    stop();
    digitalWrite(LED_BUILTIN, LOW);// led signal
    delay(2000); //delay to let inertia catch up
    digitalWrite(fanPin,LOW);

    for(int k = 0; k < 2 ; k++){
        posi[k] = 0; //reset reads on positions
        Serial.println(posi[k]); //print to show pos = 0
    }
    Serial.println("Full Reset Complete");

    //button waits to start
    do{
        for(int k = 0; k < 2; k++){//print target and current position for
monitoring
        }
    } while(digitalRead(pinON)==HIGH);

    //signal that process has started
    Serial.println("button pressed");
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);

    do{// loop until 'fire' (increase in temp) is found
        int light = lightread();
        while((lightread() < -2 || lightread() > 2) && analogRead(A7) >= thermMax)
{// rotate towards light until within % of target LIGHT SENSOR

            if (lightread() > 0) { //if light on left go left

```



```

        setMotor(-1,abs(lightread()),pinPWM[0],pinForward[0],pinBackward[0],0);
        setMotor(1,abs(lightread()),pinPWM[1],pinForward[1],pinBackward[1],1);
    }
    else if(lightread() <= 0){
        setMotor(1,abs(lightread()),pinPWM[0],pinForward[0],pinBackward[0],0);
        setMotor(-1,abs(lightread()),pinPWM[1],pinForward[1],pinBackward[1],1);
    }
    //light = lightread(); //read light again

    Serial.println(lightread()); //print to check thermistor
    Serial.print("= heat");
}
Serial.println("light source!");
stop();
while(lightread() >= -20 && lightread() <= 20 && analogRead(A7)>= thermMax)
{ //inch foward while light is in sight
    siren(500); //turn on siren
    setMotor(1,51-lightread(),pinPWM[0],pinForward[0],pinBackward[0],0);
    //go foward but adjust slightly if light strays a little
    setMotor(1,51+lightread(),pinPWM[1],pinForward[1],pinBackward[1],1);

    //light = lightread(); //read light again

    Serial.println(analogRead(A7));
    Serial.print(" = heat");
}
Serial.println("light lost"); //print to show thermistor
stop();
} while(analogRead(A7) >= thermMax);
Serial.println(analogRead(A7));
Serial.println("TOO HOT");

//whole lotta sirens...
for(int i = 0; i <= 25 ; i++){
    siren(200);
    delay(210);
}
// turn on fan
digitalWrite(fanPin,HIGH);

for(int i = 0; i <= 50 ; i++){
    siren(200);
    delay(210);
}

```

```

    //turn off fan
    digitalWrite(fanPin,LOW);

}

double mm2pulses(double d){
    double pulses;
    pulses = d * 3.42801;

    return pulses;
}

void stop(){
    for(int k = 0; k < 2; k++){
        setMotor(0,0,pinPWM[k],pinForward[k],pinBackward[k],k);
    }
    delay(500); //delay for motor inertia to catch up
}

void setMotor(int dir, int pwmVal, int pwm, int forward, int backward,int
motor){
    analogWrite(pwm,pwmVal);
    if(dir == 1){
        digitalWrite(forward,HIGH);
        digitalWrite(backward,LOW);
    }
    else if(dir == -1){
        digitalWrite(forward,LOW);
        digitalWrite(backward,HIGH);
    }
    else{
        digitalWrite(forward,LOW);
        digitalWrite(backward,LOW);
    }
}

void rotate(int c, double d){
}

template <int j,int direction>
void readEncoder(){
    //int b = digitalRead(pinEncB[j]);
    int b = 1;

```

```

    if (b > 0) {
        posi[j] += direction; // Use direction to determine increment/decrement
    } else {
        posi[j] -= direction; // Use direction to determine increment/decrement
    }
}

int lightread() {
    int value1 = analogRead(A0);
    int value2 = analogRead(A1);
    int dir = value1-value2+80;
    // Serial.println(dir);
    // Serial.print(" =Light Direction,");

    if (dir>0){
        digitalWrite(LED_BUILTIN,LOW);
    } else{
        digitalWrite(LED_BUILTIN, HIGH);
    }
    return dir;
}

//BUZZER THINGS
const int lowFreq = 950; // Low pitch in Hz
const int highFreq = 1150; // High pitch in Hz

void siren(int duration){
    long currT = millis(); //keep time
    long dT = currT-prevT;
    if (noise == 2 && dT >= duration){
        noise = 1;
        tone(A6, lowFreq, duration);
        prevT = millis();
    } else if (noise == 1 && dT >= duration){
        noise = 2;
        tone(A6, highFreq,duration);
        prevT = millis();
    }
}

```

(Optional) Firehouse Subs Fire Chief Chassis Upgrade!!!

In addition to the functional components, hats from Firehouse Subs were added to differentiate our robots and solidify them as a firefighting crew. A range of methods were used to mount the hats but the outcome remained the same in adding the hats to the top of the chassis. Some methods provided more function than others.

Discussion & Conclusion

What you would have done if there was more time?

We were successfully able to get 3 robots working. If we had more time, we could have had 4 working robots. Additionally, the robots could have been tuned better with more time to improve their functionality. Some features could have been added like adding the ability to follow each other and adding 12v for the motor instead of 8v. We could have possibly also moved from photoresistors to IR receivers to more accurately detect the infrared waves from a fire.

What worked and what didn't?

The thermistors and photoresistors worked very well as sensors, especially the thermistors which were very accurate in their temperature reading capabilities. The relay worked relatively well for controlling the motor, but it had some weird occurrences. The motors worked but the fan was ineffective in putting out fires. In the future, alternative methods may be used. Other than that, the LEDs were easy to use and worked well as emergency lights, and the buzzer was very easy to use and worked well as a simple siren.

What were the most difficult parts?

The hardest part was writing the code and going through all the iterations to get the robot to work as it did. Getting the light sensors to work in different ambient lighting (like in the lab) took a lot of testing. The actual building process came pretty fast but most of the time was spent on developing code.

What were the most successful parts?

To us, the most successful part was fire mode. It was very satisfying to see the robots activate the lights and sirens in the pursuit of a potential fire. A simple delay was used to alternate the lights and the buzzer tone and did not impede the rest of the code and its reaction time to light conditions. We did find the correct frequencies online to give the siren a classic “bee doo bee doo” siren sound.

What would your advice be for future ECEN 2270 students?

Our advice for future students would be to try to start sooner rather than later. Although our project didn't take a great deal of time, some unexpected problems popped up which led to a lot of unexpected time spent debugging. Also, debugging is a very useful skill in this course, try to take it slow when debugging and don't panic.

What are your takeaways, what did you learn?

One general takeaway we've gotten from this course is to look for resources when designing a circuit. These resources include things like the datasheets of components that you implement or insight from other people who have built circuit. We have learned how to build circuits but most importantly we have learned how to fix circuits. On the low end, we have probably spent about 30% of our time in the lab figuring out and fixing why a certain circuit does not work. We have gained an appreciation for the tools available to help with that process, mainly the oscilloscope.

Conclusion

Overall, our team successfully designed, constructed, and programmed firefighting robots capable of detecting and responding to fire hazards. Through the integration of photoresistors, thermistors, relays, and other components, we developed systems that allowed the robots to locate light sources, distinguish light sources from fire, and activate countermeasures if fire is detected. The addition of LEDs and buzzers enhanced the robots' functionality by providing emergency lights and sirens similar to those of real firetrucks. This project was not only fun but was an interesting challenge for sensor integration and programming. This project was in most aspects a success and a fitting conclusion to the semester.